# Phase 2

### Phase 2: Compiled Template Layer

*(3-4 weeks | 🛠️🟫⚡)*

| Milestone | Icon | Key Points | Portfolio Tip |
|---|---|---|---|
| **Template Compiler** | 🛠️ | - HTML → Optimized JS- Signal-aware output- Partial hydration directives | Blog: *"Compiler vs Runtime Reactivity"* |
| **DOM Optimization** | ⚡ | - Skip VDOM for direct updates- Keyed list reconciliation | Demo: *10k items benchmark vs React* |
| **Hydration Engine** | 💧 | - SSR support- Island state serialization- Client hydration guards | Case study: *SEO-friendly SPA* |

▼ **Template Compiler:**

Let's create a detailed roadmap for building **Phase 2: Compiled Template Layer** of your frontend framework, based on our decided approach: a function-based `render` with `html` and `css` tagged literals, `defineComponent` for lifecycles, and your Phase 1 reactivity system (signals, computed, effects, deep signals). I'll break this milestone into smaller tasks, using icons and explanations to guide you step-by-step, ensuring clarity and manageability. This roadmap will span **3-4 weeks**, aligning with your original timeline, and prepare you for DOM optimization and hydration (key Phase 2 goals).

---

## Phase 2: Compiled Template Layer Roadmap *(3-4 weeks | 🛠️🟫⚡)*

### Overview

- **Goal**: Build a compiled template system that transforms `html` and `css` literals into efficient DOM updates, with lifecycle support.
- **Deliverables**: A working `Counter.ts` component, compiler basics, and scoped styles.
- **Timeline**: 4 weeks, split into manageable tasks.

---

### Week 1: Template Foundation *(🛠️)*

| Task | Icon | Description | Key Steps |
|---|---|---|---|
| **1. Define Core APIs** | ⚙️ | Set up `defineComponent`, `html`, and `css` as initial mocks to shape the developer experience. | - Create `reactivity.ts` with `export function defineComponent(config) { return config; }`.<br>- Mock `html` and `css` as passthroughs: `function html(strings) { return strings.raw[0]; }`.<br>- Test with a simple `Counter.ts` file. |
| **2. Parse `html` Literal** | 🟫 | Build a basic parser to extract tags, attributes, and directives from `html`...``. | - Use `parse5` to parse the string into an AST.<br>- Log the AST for `<div><span>Count: count</span></div>`.<br>- Identify `@if`, `@for`, and event attributes. |
| **3. Generate DOM** | 🌳 | Convert the parsed `html` into static DOM nodes without reactivity. | - Write a `toDOM(ast)` function to create `document.createElement()` calls.<br>- Output a static `<div>` with children for testing.<br>- Ignore signals for now. |

**Deliverable**: A `Counter.ts` that outputs a static DOM tree from `html`...`` (no reactivity yet).

---

### Week 2: Reactivity Integration *(🔗)*

| Task | Icon | Description | Key Steps |
|---|---|---|---|

| | | | |
|---|---|---|---|
| **4. Link Signals** | ⚛️ | Connect template variables (e.g., `count`) to signals defined in `render`. | - Map signal names (e.g., `count`) to variables in `render` scope.<br>- Replace static text (e.g., `Count: count`) with initial signal values (e.g., `count()`). |
| **5. Add Effects** | 🔄 | Wrap reactive parts in `effect` calls for dynamic updates. | - For text like `count`, generate `effect(() => span.textContent = 'Count: ' + count())`.<br>- Test with `increment` updating the DOM.<br>- Handle `@input` events with setters. |
| **6. Handle Directives** | 🌀 | Implement `@if` and `@for` with reactive DOM updates. | - For `@if`, toggle `display: none` in an `effect` (e.g., `count() > 5`).<br>- For `@for`, generate a loop with `effect(() => items().forEach(...))`.<br>- Test with `items` array changes. |

**Deliverable**: A `Counter.ts` with reactive updates for `count` and `items`, using `@if` and `@for`.

## Week 3: Styles and Lifecycles (🎨🔄)

| Task | Icon | Description | Key Steps |
|---|---|---|---|
| **7. Parse `css` Literal** | 🎨 | Parse and scope styles from `css...``. | - Use `csstree` to parse CSS into an AST.<br>- Add a unique class (e.g., `.counter-xyz`) to the root ` `.<br>- Prefix selectors (e.g., `div` → `.counter-xyz div`). |
| **8. Inject Styles** | 💉 | Apply scoped styles to the DOM. | - Generate a `<style>` tag with prefixed CSS.<br>- Append to `document.head` in `render`.<br>- Test style application (e.g., blue `span`). |
| **9. Add Lifecycles** | ⏳ | Implement `mounted`, `updated`, and `destroyed` hooks in `defineComponent`. | - Return `{ mount(), update(), destroy() }` from `defineComponent`.<br>- Call `mounted` after DOM append, `updated` after `effect` runs, `destroyed` on removal.<br>- Log lifecycle events. |

**Deliverable**: A `Counter.ts` with scoped styles and lifecycle logs (e.g., "Component mounted!").

## Week 4: Polish and Optimization (⚡)

| Task | Icon | Description | Key Steps |
|---|---|---|---|
| **10. Optimize DOM** | ⚡ | Reduce redundant DOM updates for performance. | - Cache DOM nodes instead of recreating (e.g., reuse `<span>` for `count`).<br>- Benchmark updates with 1000 `items`.<br>- Use `batch` from Phase 1 for multiple signal changes. |
| **11. Error Handling** | 🐞 | Add basic error reporting for undefined signals or syntax issues. | - Check for unmatched signal names (e.g., `count` not defined).<br>- Log warnings in dev mode.<br>- Test with a broken template. |
| **12. Test Hydration** | 💧 | Lay groundwork for SSR/hydration (Phase 4 prep). | - Simulate SSR by rendering static HTML from `html...``.<br>- Write a `hydrate` function to reattach effects.<br>- Test reattaching `count` updates. |

**Deliverable**: A polished `Counter.ts` with optimized updates, error logs, and a hydration proof-of-concept.

## Timeline Visualization

Week 1: 🛠️ Template Foundation
⚙️ Define Core APIs → 📜 Parse `html` → 🌳 Generate DOM

Week 2: 🔗 Reactivity Integration
⚛️ Link Signals → 🔄 Add Effects → 🌀 Handle Directives

Week 3: 🎨🔄 Styles and Lifecycles
🎨 Parse `css` → 💉 Inject Styles → ⏳ Add Lifecycles

Week 4: ⚡ Polish and Optimization
⚡ Optimize DOM → 🐞 Error Handling → 💧 Test Hydration

### Guide with Explanations

### Week 1: Template Foundation *(🛠️)*

- **Focus**: Get the basic structure working without reactivity.
- **Why**: Establishes the compiler's input (html literals) and output (DOM nodes).
- **Tip**: Start small—render a static `<div>` first, then add children.

### Week 2: Reactivity Integration *(🔗)*

- **Focus**: Wire up your Phase 1 reactivity to make the template dynamic.
- **Why**: This is the heart of your framework—signals drive the UI.
- **Tip**: Test with a button click and array push to verify updates.

### Week 3: Styles and Lifecycles *(🎨🔄)*

- **Focus**: Add visual polish and lifecycle management.
- **Why**: Completes the developer experience with styling and hooks.
- **Tip**: Use browser dev tools to confirm style scoping works.

### Week 4: Polish and Optimization *(⚡)*

- **Focus**: Refine performance and prepare for future phases.
- **Why**: Ensures scalability and avoids regret (e.g., hydration support).
- **Tip**: Benchmark against Angular/React for DOM updates to validate speed.

---

### Things to Keep in Mind

- **Modularity**: Write the compiler as separate modules (parser, generator) for future extensibility.
- **Testing**: Test each task with a simple `Counter.ts` to catch issues early.
- **Docs**: Note decisions (e.g., why `@if` over `ngIf`) for Phase 5 documentation.

This roadmap breaks Phase 2 into 12 actionable tasks, each building on the last, to deliver a functional compiled template layer in 4 weeks. You'll have a working system by Week 3, with Week 4 for polish. What do you think—ready to start with Week 1? Let me know if you want to adjust any tasks!

## ▼ HTML TEMPLATE

Great! Based on the folder structure you provided and the current progress in **Phase 2: Compiled Template Layer**, we can build a detailed roadmap to continue developing the compiler. Your existing work includes a solid tokenizer (`tokenizer.ts`) and parser (`parser.ts`) in the `core/compiler/reactivity/internals` directory, along with a `signalGraph.ts` for reactivity management. This gives us a strong foundation to expand into the compiled template layer, focusing on the **Template Compiler** milestone (🛠️) as outlined in your Phase 2 plan.

Given your folder structure, new files should be saved under `core/compiler/phase2-compiler` (since you're working on Phase 2) or `core/compiler/reactivity/internals` if they extend the existing reactivity internals. We'll also consider creating utility files in `core/utils` if needed.

### Phase 2: Compiled Template Layer Roadmap *(3-4 weeks | 🛠️📜⚡)*

### Overview

- **Goal**: Develop a compiler that transforms `html` and `css` tagged literals into optimized JavaScript code, integrating with your Phase 1 reactivity system (signals, effects, etc.) and preparing for DOM optimization and hydration.

- **Deliverables**: A working compiler with `html` and `css` support, a sample `Counter` component, and integration with the tokenizer/parser.
- **Timeline**: 4 weeks, broken into manageable tasks.

## Week 1: Compiler Setup and HTML Parsing (🛠️)

| Task | Icon | Description | Key Steps | File Location |
|---|---|---|---|---|
| **1. Set Up Compiler Entry Point** | ⚙️ | Create the main compiler file to orchestrate the process. | - Create `compiler.ts` in `core/compiler/phase2-compiler`.<br>- Export a `compile` function that takes `html` and `css` literals.<br>- Integrate with `Tokenizer` and `Parser` from `internals`. | `core/compiler/phase2-compiler/compiler.ts` |
| **2. Enhance HTML Parser Output** | 📜 | Extend the parser to handle tagged literal syntax (e.g., `html`...``). | - Modify `parser.ts` to recognize signal placeholders (e.g., `{{ count }}`) and directives (e.g., `@click`).<br>- Add support for extracting attribute signals and events.<br>- Test with a simple `<div>{{ count }}</div>`. | `core/compiler/reactivity/internals/parser.ts` |
| **3. Generate Initial JS Output** | 🌳 | Convert parsed AST into basic JavaScript code. | - Add a `codegen.ts` file to generate static JS from the AST.<br>- Output `document.createElement` calls for static tags.<br>- Log the generated code for `<div>{{ count }}</div>`. | `core/compiler/phase2-compiler/codegen.ts` |

**Deliverable**: A `compiler.ts` that parses and generates static JS from a simple `html` literal, tested with a basic template.

## Week 2: Reactivity and Directive Integration (🔗)

| Task | Icon | Description | Key Steps | File Location |
|---|---|---|---|---|
| **4. Integrate Signals** | ⚛️ | Link signal placeholders (e.g., `{{ count }}`) to the reactivity system. | - Update `codegen.ts` to replace `{{ signal }}` with `effect` calls (e.g., `effect(() => node.textContent = count())`).<br>- Map signal names to the `signalGraph` bindings.<br>- Test with a `Counter` component updating on click. | `core/compiler/phase2-compiler/codegen.ts` |
| **5. Implement Directives** | 🌀 | Add support for `@if`, `@for`, and event directives (e.g., `@click`). | - Extend `parser.ts` to parse directives into the AST.<br>- Generate conditional logic (e.g., `if (count() > 0) {...}`) and loops (e.g., `items().forEach(...)`) in `codegen.ts`.<br>- Test with `@if` and `@for` in a template. | `core/compiler/reactivity/internals/parser.ts`<br>`core/compiler/codegen.ts` |
| **6. Add Event Handling** | 🔌 | Connect event directives to signal setters. | - Update `codegen.ts` to generate event listeners (e.g., `node.addEventListener('click', () => setCount(count() + 1))`).<br>- Link to `signalGraph` for event bindings.<br>- Test with a button incrementing a counter. | `core/compiler/phase2-compiler/codegen.ts` |

**Deliverable**: A `Counter` component with reactive updates, `@if`/`@for` directives, and event handling.

## Week 3: CSS Handling and Optimization (🎨⚡)

| Task | Icon | Description | Key Steps | File Location |
|---|---|---|---|---|
| **7. Parse and Scope CSS** | 🎨 | Process `css` literals and apply scoping. | - Create `cssProcessor.ts` to parse CSS using a library (e.g., `csstree`).<br>- Add unique class scoping (e.g., `.counter-xyz`) to selectors.<br>- Test with a simple `css` ...`` block. | `core/compiler/phase2-compiler/cssProcessor.ts` |
| **8. Inject Scoped Styles** | 🖌️ | Generate and append styled `<style>` tags. | - Update `compiler.ts` to include a `style` injection function.<br>- Append scoped CSS to `document.head`.<br>- Test styling a `Counter` component (e.g., blue text). | `core/compiler/phase2-compiler/compiler.ts` |
| **9. Optimize Code Generation** | ⚡ | Reduce overhead in the generated JS. | - Add caching for repeated DOM nodes in `codegen.ts`.<br>- Minimize `effect` calls by batching updates.<br>- Benchmark with 100 static elements. | `core/compiler/phase2-compiler/codegen.ts` |

**Deliverable**: A `Counter` component with scoped CSS and optimized JS output.

## Week 4: Testing and Hydration Prep (🐞💧)

| Task | Icon | Description | Key Steps | File Location |
|---|---|---|---|---|
| **10. Unit Tests** | 🐞 | Write tests for the compiler pipeline. | - Create `compiler.test.ts` with Jest to test `compile` output.<br>- Test parsing, codegen, and CSS scoping.<br>- Cover edge cases (e.g., invalid signals). | `core/compiler/phase2-compiler/compiler.test.ts` |
| **11. Error Handling** | ⚠️ | Add robust error reporting. | - Update `parser.ts` and `codegen.ts` to throw meaningful errors (e.g., undefined signals).<br>- Log warnings in dev mode.<br>- Test with a broken template. | `core/compiler/reactivity/internals/parser.ts`<br>`core/compiler/phase2-compiler/codegen.ts` |
| **12. Hydration Foundation** | 💧 | Prepare for SSR/hydration support. | - Add a `hydrate.ts` file to reattach effects to pre-rendered DOM.<br>- Simulate SSR by generating static HTML | `core/compiler/phase2-compiler/hydrate.ts` |

| | | | | |
|---|---|---|---|---|
| | | | from `compile`. <br>- Test rehydrating a `Counter` component. | |

**Deliverable**: A tested compiler with error handling and a hydration proof-of-concept.

---

## Timeline Visualization

Week 1: 🛠️ Compiler Setup and HTML Parsing
⚙️ Set Up Compiler → 📜 Enhance Parser → 🌳 Generate JS

Week 2: 🔗 Reactivity and Directive Integration
⚛️ Integrate Signals → 🌀 Implement Directives → 🔊 Add Events

Week 3: 🎨⚡ CSS Handling and Optimization
🎨 Parse CSS → 💉 Inject Styles → ⚡ Optimize Code

Week 4: 🐞💧 Testing and Hydration Prep
🐞 Unit Tests → ⚠️ Error Handling → 💧 Hydration Foundation

---

## Guide with Explanations

### Week 1: Compiler Setup and HTML Parsing (🛠️)

- **Focus**: Establish the compiler's structure and leverage your existing tokenizer/parser.
- **Why**: Builds on your current work to create a foundation for dynamic templates.
- **Tip**: Start with a minimal `compile` function and expand it gradually.

### Week 2: Reactivity and Directive Integration (🔗)

- **Focus**: Tie the compiler to your Phase 1 reactivity system.
- **Why**: This is where your framework's reactivity shines, enabling dynamic UIs.
- **Tip**: Test each directive separately to isolate issues.

### Week 3: CSS Handling and Optimization (🎨⚡)

- **Focus**: Add styling and improve performance.
- **Why**: Completes the developer experience and sets up DOM optimization.
- **Tip**: Use browser dev tools to verify CSS scoping.

### Week 4: Testing and Hydration Prep (🐞💧)

- **Focus**: Ensure reliability and future-proof the system.
- **Why**: Prepares you for Phase 4 (hydration) and avoids technical debt.
- **Tip**: Benchmark hydration against a static render to validate efficiency.

---

## Things to Keep in Mind

- **Modularity**: Keep `parser.ts`, `codegen.ts`, and `cssProcessor.ts` as separate concerns for reusability.
- **Testing**: Use your existing `.test.ts` pattern to validate each step.
- **Documentation**: Add comments in new files for Phase 5 docs.

This roadmap builds on your current solutions and aligns with your Phase 2 goals. Let's start with **Week 1, Task 1: Set Up Compiler Entry Point**. Would you like to proceed with creating `compiler.ts`, or do you want to adjust anything first?